អំពីប្លង់វេបសាយ

ការអភិវឌ្ឍវេបសាយទំនើបតម្រូវឱ្យយើងយល់ដឹងអំពីរចនាសម្ព័ន្ធ HTML និងប្លង់ (Layout) ដ៏ត្រឹមត្រូវ។ ឯកសារនេះនឹងណែនាំអ្នកតាមរយៈគំនិតសំខាន់ៗ ចំនួនបី គឺ **ធាតុ Semantic**, **ធាតុ Non-Semantic**, និង **ប្លង់ផ្សេងៗ** ដែលប្រើក្នុងការរចនាវេបសាយ។ ចូរប្រើប្រាស់ឯកសារនេះជាមគ្គុទ្ទេសក៍ក្នុងការរៀនសូត្រ និងអភិវឌ្ឍជំនាញ HTML/CSS របស់អ្នក។

មាតិកា

ឯកសារនេះត្រូវបានបែងចែកជាមេរៀនសំខាន់ៗ ដើម្បីជួយឱ្យអ្នករៀនយ៉ាងជាប់លាប់ និងយល់ដឹងច្បាស់លាស់អំពីប្លង់វេបសាយ។

01

ធាតុ Semantic

ធាតុ HTML ដែលមានន័យច្បាស់លាស់ ងាយយល់ទាំងមនុស្ស និងម៉ាស៊ីន

02

ធាតុ Non-Semantic

ធាតុ HTML ទូទៅ ដែលមិនមានន័យពិសេស ប្រើសម្រាប់រចនា

03

អំពីប្លង់នៃវេបសាយ

ប្រភេទប្លង់ផ្សេងៗ Fixed, Fluid, Responsive, Single Page, Grid

មេរៀនទី ១

អំពីធាតុ Semantic

ធាតុ **Semantic** គឺជាធាតុ HTML ដែលឈ្មោះ Tag របស់វាបង្ហាញពីន័យ និងតួនាទីច្បាស់លាស់នៃមាតិកា។ ពាក្យ Semantic មានន័យថា "ទាក់ទងនឹងន័យ" ដូច្នេះ Tag Semantic ជួយឱ្យ Browser, Search Engine និង Screen Reader យល់ពីរចនាសម្ព័ន្ធផ្នែកក្នុងនៃទំព័រ HTML។

ឧទាហរណ៍ ការប្រើ `<header>` ជំនួស `<div id="header">` ធ្វើឱ្យកូដកាន់តែច្បាស់ និងងាយស្រួលអាន។ ធាតុ Semantic ត្រូវបានណែនាំជាផ្លូវការក្នុង HTML5 ដើម្បីធ្វើឱ្យគំរូ (structure) នៃទំព័រវេបមានន័យ ច្បាស់ជាង HTML ចាស់។

គុណប្រយោជន៍

- ធ្វើឱ្យ SEO ប្រសើរជាង
- ងាយអានកូដ
- ជួយ Accessibility
- រចនាសម្ព័ន្ធត្រឹមត្រូវ

Tag Semantic ទូទៅ

- `<header>` – ផ្នែកស្បែក
- `<nav>` – ផ្នែកនាំផ្លូវ
- `<main>` – មាតិកាចម្បង
- `<section>` – ផ្នែកមួយ
- `<article>` – អត្ថបទ
- `<aside>` – ផ្នែកចំហៀង
- `<footer>` – ផ្នែកបាត

ការប្រើ Semantic Elements ទៀងទាត់ក្នុងគម្រោងរបស់អ្នក នឹងជួយបង្កើនគុណភាពនៃ Code Base ជួយ Search Engine ស្វែងរកទំព័ររបស់អ្នក ហើយក៏ជួយឱ្យ Developer ផ្សេងៗអាចយល់ ហើយថែទាំ Code បានងាយស្រួល។ ការប្រើ Semantic ក៏ជាការអនុវត្ត Best Practice ក្នុងការសរសេរ HTML ទំនើបផងដែរ។

SEO ប្រសើរ

Google និង Search Engine ផ្សេងៗអានរចនាសម្ព័ន្ធ Semantic ហើយចាត់ជាន់ (rank) ទំព័ររបស់អ្នកបានត្រឹមត្រូវ

Accessibility

Screen Reader ប្រើ Semantic Tags ដើម្បីជួយអ្នកប្រើប្រាស់ពិការភ្នែក ស្តាប់ទំព័ររបស់អ្នកបានត្រឹមត្រូវ

Code ងាយស្រួល

ក្រុមការងារ Developer អានកូដ Semantic បានលឿន ធ្វើឱ្យការថែទាំ Project ក្លាយជារឿងងាយ

មេរៀនទី ២

អំពីធាតុ Non-Semantic

ធាតុ **Non-Semantic** គឺជា Tag HTML ដែលមិនបង្ហាញពីន័យ ឬតួនាទីនៃមាតិកាដែលវាផ្ទុក។ Tag ទាំងនេះត្រូវបានប្រើសំខាន់ ដើម្បីបង្កើតរចនាសម្ព័ន្ធ (structure) ឬក្រុម (group) ធាតុ HTML ដោយមានការប្រើ CSS ដើម្បីធ្វើ Style។

Tag Non-Semantic ដ៏ពេញនិយមបំផុតគឺ `<div>` និង `<span>`។ ទោះបីជា Tag ទាំងនេះមិនមានន័យ Semantic ពិសេស ក៏ដោយ ក៏ពួកវានៅតែមានតម្លៃជាពិសេសសម្រាប់ការរចនា Layout ស្មុគ្រស្មាញ ដែល Semantic Tags មិនអាចគ្របដណ្តប់ទាំងស្រុង។

Tag Non-Semantic ទូទៅ

- `<div>` – Block-level container
- `<span>` – Inline container

ការប្រើប្រាស់

- បង្កើត Layout ស្មុគ្រស្មាញ
- ក្រុម Elements ដើម្បីធ្វើ Style
- បន្ថែម Class ឬ ID សម្រាប់ JavaScript

ប្រៀបធៀប Semantic vs Non-Semantic

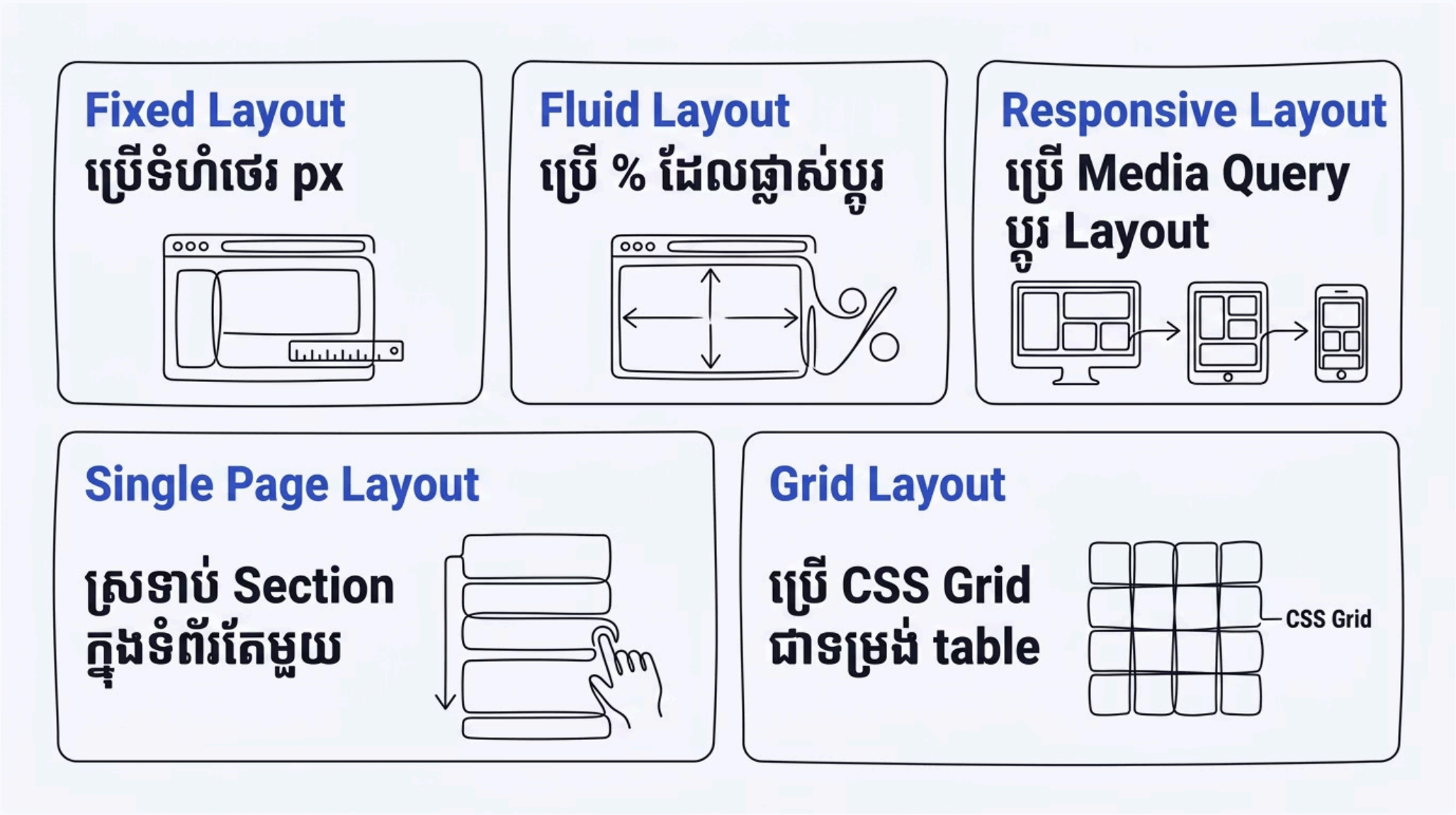
- Semantic – ន័យច្បាស់ ល្អសម្រាប់ SEO
- Non-Semantic – ឥតន័យ ប្រើសម្រាប់ Style
- ក្នុងការអភិវឌ្ឍ ប្រើ Semantic ជាដំបូង
- ប្រើ Non-Semantic ក្នុងករណីពិសេស

ការប្រើ `<div>` ច្រើនហួសប្រមាណ ត្រូវបានគេហៅថា "Div Soup" ដែលធ្វើឱ្យ Code ពិបាកអាន និងថែទាំ។ ការប្រើ Semantic Tags ជំនួស Div ក្នុងកន្លែងដែលសមគួរ ជាការអនុវត្តដ៏ល្អបំផុត។

មេរៀនទី ៣

អំពីប្លង់នៃវេបសាយ

ប្លង់ (Layout) គឺជារបៀបដែលមាតិកា ត្រូវបានរៀបចំ និងបង្ហាញក្នុងទំព័រវេបសាយ។ ការជ្រើសប្លង់ត្រឹមត្រូវគឺសំខាន់ណាស់ ដើម្បីធានាឱ្យទំព័ររបស់អ្នកមើលទៅល្អ ប្រើប្រាស់ងាយស្រួល ហើយដំណើរការបានល្អ នៅលើឧបករណ៍ផ្សេងៗ។



ប្រភេទប្លង់នីមួយៗមានចំណុចខ្លាំង និងខ្សោយ ហើយការជ្រើសប្រើប្រព័ន្ធត្រឹមត្រូវទៅតាមគោលបំណងគម្រោង គឺជាជំហានសំខាន់ក្នុងការរៀបចំ Project HTML/CSS យ៉ាងជោគជ័យ។

ប្លង់ ៣.១

អំពី Fixed Layout

Fixed Layout គឺប្រភេទប្លង់ ដែលកំណត់ទំហំទទឹង (width) ជាកម្រិតថេរ ដោយប្រើ Unit `px` (Pixel)។ មិនថា Browser Window ធំ ឬតូច ខ្លឹមសារ Layout នឹងមានទំហំថេរ ចុះ-ឡើងមិនផ្លាស់ប្តូរ។ ផ្ទៃទំព័រ (Canvas) ត្រូវបានចងភ្ជាប់នៅ Width មួយ ដូចជា `960px` ឬ `1200px`។

Fixed Layout ត្រូវបានប្រើច្រើនក្នុង Website ចាស់ៗ នៅក្នុងសម័យដែល Mobile Device មិនទាន់ពេញនិយម។ ទោះបីជាការប្រើ Fixed Layout ងាយស្រួលក្នុងការគ្រប់គ្រង Design ក៏ដោយ ក៏ Layout នេះមានបញ្ហាក្នុងការប្រើ នៅលើ Screen ទំហំតូចដូចជា Smartphone ដោយព្រោះ User ត្រូវ Scroll ទំហំ Horizontal ដើម្បីមើលមាតិកាទាំងស្រុង។

✅ ចំណុចខ្លាំង

- ងាយគ្រប់គ្រង Design
- ទំព័របង្ហាញដូចគ្នា Browser ណាក៏ដោយ
- ល្អសម្រាប់ Print Layout

❌ ចំណុចខ្សោយ

- មើលមិនល្អ Screen តូច
- User ត្រូវ Scroll Horizontal
- មិនល្អ SEO Mobile

📌 ករណីប្រើ

- Website Desktop Only
- Print Stylesheets
- Email Templates

CSS Property ដែលប្រើក្នុង Fixed Layout រួមមាន `width: 960px`, `margin: 0 auto` (ដើម្បីបង្ហាញ Layout នៅចំកណ្តាល) ហើយ Container ចម្បងជាទូទៅ ត្រូវបានចង Fixed Width ហើយ Elements ខាងក្នុង Arranged ទៅ Percentage ឬ Pixel អាស្រ័យ Design។

ឧទាហរណ៍ – FIXED LAYOUT

ឧទាហរណ៍ Fixed Layout ជាមួយ Semantic

ខាងក្រោមនេះ គឺជាឧទាហរណ៍ HTML ដែលប្រើ **Semantic Tags** ដើម្បីបង្កើត Fixed Layout ។ ការប្រើ `<header>`, `<nav>`, `<main>`, `<aside>`, `<footer>` ជួយបង្ហាញរចនាសម្ព័ន្ធច្បាស់លាស់ ងាយអាន ហើយ SEO ប្រសើរ។

```
<!-- HTML Structure — Fixed Layout with Semantic Tags -->
<body>
  <div class="wrapper">

    <header>
      <h1>Logo នៃ Website</h1>
    </header>

<nav>
  <ul>
    <li><a href="#">ទំព័រដើម</a></li>
    <li><a href="#">អំពីយើង</a></li>
    <li><a href="#">ទំនាក់ទំនង</a></li>
  </ul>
</nav>

<main>
  <article>
    <h2>ចំណងជើងអត្ថបទ</h2>
    <p>មាតិកាចម្បង...</p>
  </article>
  <aside>
    <h3>ផ្នែកចំហៀង</h3>
      <p>ព័ត៌មានបន្ថែម...</p>
     </aside>
    </main>

    <footer>
      <p>&copy; 2024 Website Name</p>
    </footer>

  </div>
</body>

/* CSS — Fixed Layout */
.wrapper {
  width: 960px;
  margin: 0 auto;
}
header, nav, footer {
  width: 100%;
}
main {
  display: flex;
  gap: 20px;
}
article { width: 70%; }
aside   { width: 30%; }
```

ចំណាំ: `.wrapper` ត្រូវបាន Set `width: 960px` ដែលជា Fixed Width ហើយ `margin: 0 auto` ធ្វើឱ្យ Layout ស្ថិតនៅចំកណ្តាល Screen ។ ធាតុ Semantic ជួយបង្ហាញរចនាសម្ព័ន្ធ ច្បាស់ ដោយ Browser ហើយ Search Engine អានបានត្រូវ។

ឧទាហរណ៍ – FIXED LAYOUT

ឧទាហរណ៍ Fixed Layout ជាមួយ Non-Semantic

ខាងក្រោម គឺជាឧទាហរណ៍ HTML ដែលប្រើ **Non-Semantic Tags** (`<div>`) ដើម្បីបង្កើត Fixed Layout ។ Structure ដូចគ្នា ប៉ុន្តែ Tags ខ្វះន័យ Semantic ដែលធ្វើឱ្យ Code ពិបាកអានជាង ដូចជាករណី "Div Soup"។

```
<!-- HTML Structure — Fixed Layout with Non-Semantic Tags -->
<body>
  <div class="wrapper">

    <div class="header">
      <h1>Logo នៃ Website</h1>
    </div>

<div class="nav">
  <ul>
    <li><a href="#">ទំព័រដើម</a></li>
    <li><a href="#">អំពីយើង</a></li>
    <li><a href="#">ទំនាក់ទំនង</a></li>
  </ul>
</div>

<div class="main">
  <div class="article">
    <h2>ចំណងជើងអត្ថបទ</h2>
    <p>មាតិកាចម្បង...</p>
  </div>
  <div class="aside">
      <h3>ផ្នែកចំហៀង</h3>
      <p>ព័ត៌មានបន្ថែម...</p>
    </div>
  </div>

    <div class="footer">
      <p>&copy; 2024 Website Name</p>
    </div>

  </div>
</body>

/* CSS — Fixed Layout */
.wrapper {
  width: 960px;
  margin: 0 auto;
}
.main {
  display: flex;
  gap: 20px;
}
.article { width: 70%; }
.aside   { width: 30%; }
```

Semantic (ល្អ)

ប្រើ `<header>`, `<nav>`, `<main>` ដែល Browser, SEO, Screen Reader យល់ថ្លែ

Non-Semantic (ត្រូវដឹង)

ប្រើ `<div class="header">` ជំនួស ដែល Browser មើលឃើញ div ធម្មតា មិនដឹងថាជា header

ប្លង់ ៣.២

អំពី Fluid / Liquid Layout

Fluid Layout ឬ **Liquid Layout** គឺប្រភេទប្លង់ ដែលទំហំ Width ត្រូវបានគណនាជា Percentage (%) ជំនួស Pixel (px) ។ នេះមានន័យថា Layout នឹងពង្រីក ឬ ច្រួតចុះ ដោយស្វ័យប្រវត្តិ ទៅតាម ទំហំ Browser Window របស់ User ។

ឧទាហរណ៍ ប្រសិនបើ Container ត្រូវ Set `width: 80%` នៅ Screen ទំហំ 1200px Container នឹងមានទំហំ 960px ប៉ុន្តែ Screen ទំហំ 800px Container នឹងមានទំហំ 640px ដោយស្វ័យប្រវត្តិ។ ការ Scale នេះ ធ្វើឱ្យ Layout មើលទៅ "ងាយ" ជាង Fixed ប៉ុន្តែ គ្មាន Control ខ្លះ ដូចជា Text អាច Wrap ខុស ឬ Images ចំហៀងអាចលូតធំ ឬ Crush ។

1

Screen ធំ (1400px)

Container 80% = 1120px – Layout ច្បាស់ ស្អាត

2

Screen មធ្យម (900px)

Container 80% = 720px – Layout Scale ចុះ ស្វ័យប្រវត្តិ

3

Screen តូច (400px)

Container 80% = 320px – Layout Squeeze ចូល អាចតូចពេក

Fluid Layout ល្អ ជំហានចូលដំណើរ Responsive Design ប៉ុន្តែ ពុំមាន Breakpoints ដូច Responsive Layout ពិតប្រាកដ ទេ។ ការប្រើ Fluid Layout ជួនកាល ត្រូវការ `min-width` និង `max-width` ដើម្បីទប់ Layout មិនឱ្យ Scale ក្រៅកំណត់ ។ Properties ដែលប្រើច្រើន គឺ `width: 90%`, `max-width: 1200px`, `margin: 0 auto` ។

ឧទាហរណ៍ – FLUID LAYOUT

ឧទាហរណ៍ Fluid Layout ជាមួយ Semantic

ខាងក្រោម គឺជាឧទាហរណ៍ **Fluid Layout** ដែលប្រើ Semantic Tags ។ ចំណុចខ្លាំង គឺ Width ត្រូវ Set ជា Percentage ធ្វើឱ្យ Layout Scale ដោយស្វ័យប្រវត្តិ ។ ប្រើ `max-width` ដើម្បីការពារ Layout មិនឱ្យ Stretch ហួស Screen ធំ ។

```
<!-- HTML Structure — Fluid Layout with Semantic Tags -->
<body>
  <div class="container">

    <header>
      <h1>Website Title</h1>
    </header>

<nav>
  <ul>
    <li><a href="#">ទំព័រដើម</a></li>
    <li><a href="#">ផលិតផល</a></li>
    <li><a href="#">ទំនាក់ទំនង</a></li>
  </ul>
</nav>

<main>
  <section class="content">
    <h2>មាតិកាចម្បង</h2>
    <p>ខ្លឹមសារ...</p>
  </section>
  <aside class="sidebar">
      <h3>Sidebar</h3>
      <p>ព័ត៌មានបន្ថែម...</p>
    </aside>
  </main>

  <footer>
    <p>&copy; 2024</p>
  </footer>

  </div>
</body>

/* CSS — Fluid Layout */
.container {
  width: 90%;
  max-width: 1200px;
  margin: 0 auto;
}
main {
  display: flex;
  gap: 2%;
}
.content { width: 70%; }
.sidebar { width: 28%; }
```

គន្លឹះ៖ ការប្រើ `max-width: 1200px` ធ្វើឱ្យ Layout Scale ឡើង ប៉ុន្តែ នៅពេល Screen ធំជាង 1200px Layout នឹង Stop ពង្រីក ។ ធ្វើឱ្យ Design ស្អាត Screen ធំ ហើយ Fluid Screen តូច ។

ឧទាហរណ៍ – FLUID LAYOUT

ឧទាហរណ៍ Fluid Layout ជាមួយ Non-Semantic

ឧទាហរណ៍ខាងក្រោម ប្រើ **Non-Semantic Tags** (`<div>`) ដើម្បីបង្កើត Fluid Layout ដូចគ្នា ប៉ុន្តែ គ្រាន់តែជំនួស Semantic Tags ដោយ Div ។ CSS Logic ដូចគ្នាទាំងស្រុង ប៉ុន្តែ Structure ខ្វះ Meaning ។

```
<!-- HTML Structure — Fluid Layout with Non-Semantic Tags -->
<body>
  <div class="container">

    <div class="header">
      <h1>Website Title</h1>
    </div>

    <div class="nav">
      <ul>
        <li><a href="#">ទំព័រដើម</a></li>
        <li><a href="#">ផលិតផល</a></li>
        <li><a href="#">ទំនាក់ទំនង</a></li>
      </ul>
    </div>

    <div class="main">
      <div class="content">
        <h2>មាតិកាចម្បង</h2>
        <p>ខ្លឹមសារ...</p>
      </div>
      <div class="sidebar">
        <h3>Sidebar</h3>
        <p>ព័ត៌មានបន្ថែម...</p>
      </div>
    </div>

    <div class="footer">
      <p>&copy; 2024</p>
    </div>

  </div>
</body>

/* CSS — Fluid Layout (Non-Semantic) */
.container {
  width: 90%;
  max-width: 1200px;
  margin: 0 auto;
}
.main {
  display: flex;
  gap: 2%;
}
.content { width: 70%; }
.sidebar { width: 28%; }
```

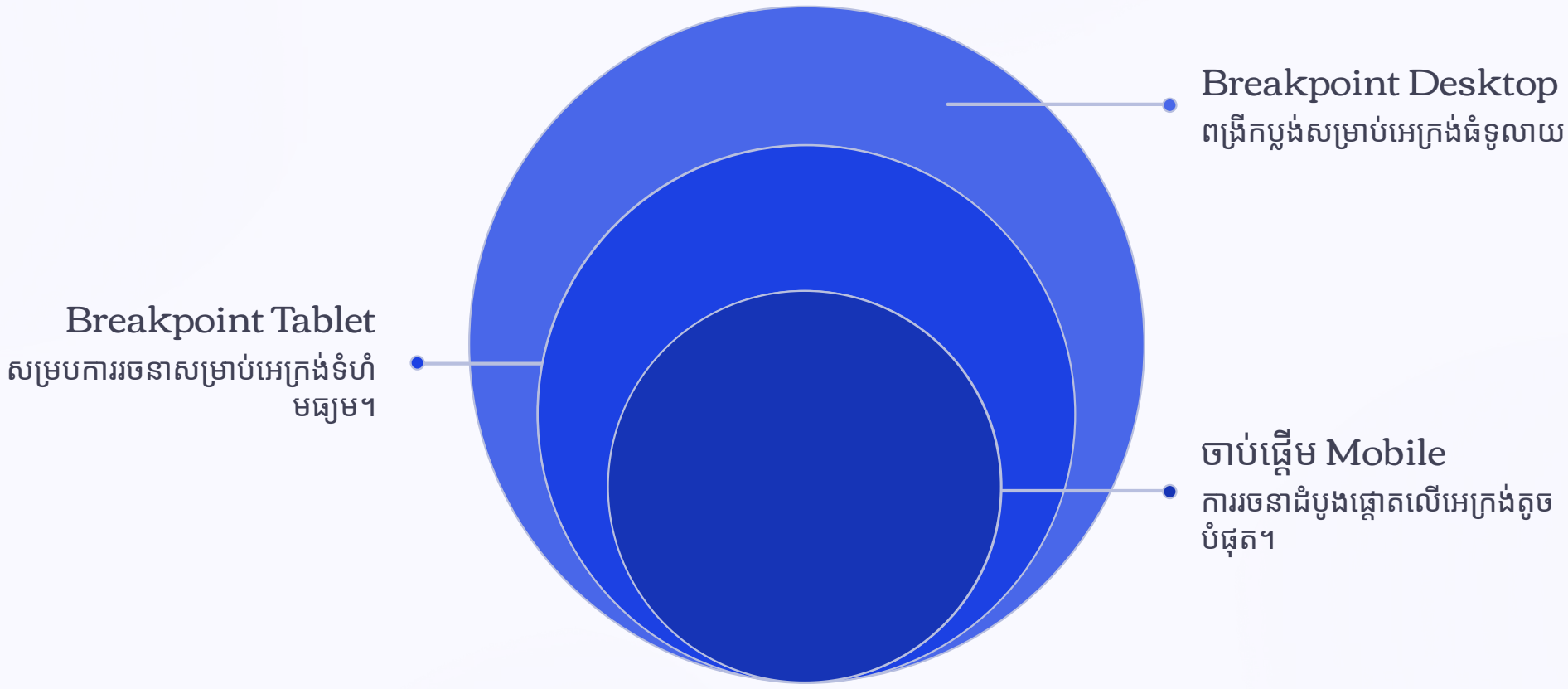
ឃើញថា CSS ដូចគ្នា ២ករណី ។ ភាពខុសគ្នា គ្រាន់តែ Tag Name HTML ប៉ុណ្ណោះ ។ ក្នុងការ Practice ជា Developer ល្អ គួរជ្រើស Semantic Tags ជាជម្រើសដំបូង ។

ប្លង់ ៣.៣

អំពី Responsive Layout

Responsive Layout គឺប្លង់ ដែល Design ផ្លាស់ប្តូរ (Adapt) ដោយស្វ័យប្រវត្តិ ទៅ Breakpoints ផ្សេងៗ ដោយប្រើ **CSS Media Queries** ។ នេះមានន័យថា Layout នៅ Desktop, Tablet, Mobile Phone អាចខុសគ្នា ទាំងស្រុង ដោយ CSS ។ Responsive Design ក្លាយជា Standard ក្នុង Web Development ទំនើប ។

គំនិតចម្បង គឺ **Mobile First** ដែលរចនា Layout Mobile ដំបូង ហើយ Scale Up ទៅ Tablet, Desktop ។ ការប្រើ `@media` Rules ក្នុង CSS ជូន Browser ជ្រើស Style ត្រូវ ។ Units ដែលប្រើ រួម `%`, `vw`, `em`, `rem` ។



Responsive Layout ក្លាយជា Best Practice ក្នុង Industry ព្រោះ Google ប្រើ Mobile-First Indexing ។ Web ដែល Responsive SEO ប្រសើរ User Experience ល្អ ។ Framework ដូចជា Bootstrap, Tailwind CSS ច្រើន Built-in Responsive System ។

Mobile

⩽ 576px – Column តែមួយ Stack ចុះ

Tablet

⩾ 768px – ២ Columns Side by Side

Desktop

⩾ 1200px – Layout ពេញ ទំហំ

ឧទាហរណ៍ – RESPONSIVE LAYOUT

ឧទាហរណ៍ Responsive Layout ជាមួយ Semantic

ឧទាហរណ៍ខាងក្រោម ប្រើ **Semantic Tags** ជាមួយ **CSS Media Queries** ដើម្បីបង្កើត Responsive Layout ។ Layout ផ្លាស់ប្តូរ ពី Stack Vertical នៅ Mobile ទៅ Side-by-Side នៅ Desktop ។

```
<!-- HTML — Responsive Layout with Semantic Tags -->
<body>
  <header>
    <h1>Website Title</h1>
    <nav>
      <ul>
        <li><a href="#">ទំព័រដើម</a></li>
        <li><a href="#">Blog</a></li>
        <li><a href="#">ទំនាក់ទំនង</a></li>
      </ul>
    </nav>
</header>

<main class="layout">
  <article class="content">
    <h2>ចំណងជើងអត្ថបទ</h2>
    <p>មាតិកាចម្បង...</p>
  </article>
  <aside class="sidebar">
    <h3>Sidebar</h3>
    <p>ព័ត៌មានបន្ថែម...</p>
  </aside>
</main>

<footer>
  <p>&copy; 2024</p>
</footer>
</body>

/* CSS — Mobile First Responsive */
.layout {
  display: flex;
  flex-direction: column; /* Mobile: Stack ចុះ */
}
.content { width: 100%; }
.sidebar { width: 100%; }

/* Tablet ≥ 768px */
@media (min-width: 768px) {
  .layout {
    flex-direction: row;
    gap: 20px;
  }
  .content { width: 65%; }
  .sidebar { width: 33%; }
}

/* Desktop ≥ 1200px */
@media (min-width: 1200px) {
  .layout {
    max-width: 1200px;
    margin: 0 auto;
  }
  .content { width: 70%; }
  .sidebar { width: 28%; }
}
```

- **Mobile First:** ចាប់ផ្តើម Style Mobile ជាមុន ហើយ Override ឡើង Tablet, Desktop ។ Approach នេះ ល្អ Performance ព្រោះ Mobile Load CSS តិចជាង ។

ឧទាហរណ៍ – RESPONSIVE LAYOUT

ឧទាហរណ៍ Responsive Layout ជាមួយ Non-Semantic

ឧទាហរណ៍ Responsive Layout ប្រើ **Non-Semantic Div Tags** ។ Logic CSS Media Query ដូចគ្នា ប៉ុន្តែ HTML ប្រើ `<div>` ទាំងអស់ ។

```
<!-- HTML — Responsive Layout with Non-Semantic Tags -->
<body>
  <div class="header">
    <h1>Website Title</h1>
    <div class="nav">
      <ul>
        <li><a href="#">ទំព័រដើម</a></li>
        <li><a href="#">Blog</a></li>
        <li><a href="#">ទំនាក់ទំនង</a></li>
      </ul>
    </div>
  </div>

  <div class="layout">
    <div class="content">
      <h2>ចំណងជើងអត្ថបទ</h2>
      <p>មាតិកាចម្បង...</p>
    </div>
    <div class="sidebar">
      <h3>Sidebar</h3>
      <p>ព័ត៌មានបន្ថែម...</p>
    </div>
  </div>

  <div class="footer">
    <p>&copy; 2024</p>
  </div>
</body>

/* CSS — Responsive (Non-Semantic) */
.layout {
  display: flex;
  flex-direction: column;
}
.content, .sidebar { width: 100%; }

@media (min-width: 768px) {
  .layout {
    flex-direction: row;
    gap: 20px;
  }
  .content { width: 65%; }
  .sidebar { width: 33%; }
}
```

ចំណុចសំខាន់

CSS Media Query Logic ដូចគ្នា មិនខ្វល់ Semantic ឬ Non-Semantic ប៉ុន្តែ Semantic ធ្វើ HTML Structure ប្រសើរ

ពេលណាប្រើ Non-Semantic?

ក្នុង Framework ដូចជា React, Vue Component ជួនកាលប្រើ `<div>` ជា Wrapper ក្នុង Template Logic ដែលជៀសមិនបាន

ប្លង់ ៣.៤

អំពី Single Page Layout

Single Page Layout ឬ **One Page Website** គឺប្រភេទ Design ដែល Content ទាំងអស់ ស្ថិតក្នុង HTML File តែមួយ ។ User Scroll ចុះដើម្បីមើល Section ផ្សេងៗ ជំនួសការ Click ចូល Page ថ្មី ។ ការ Navigate ចូល Section ជួនកាលប្រើ Anchor Links (`#section-id`) ។

Single Page Layout ពេញនិយម Portfolio Website, Landing Page, Product Showcase ។ ការប្រើ `scroll-behavior: smooth` ក្នុង CSS ជួយ Animate ការ Scroll ឱ្យ ស្អាត ។ ឧបករណ៍ Modern ជំនួស ដូចជា **SPA (Single Page Application)** ប្រើ JavaScript Framework (React, Vue, Angular) ដើម្បីផ្ទុក Content ថាមវន្ត (Dynamic) ។

📌 Hero Section

ផ្នែក Banner ធំ នៅ Top ជាមួយ Call-to-Action ជាប្រភេទ Full Height ។

📋 About Section

ផ្នែកណែនាំ ឬ Story អំពី Brand ឬ Company ។

🛠️ Services Section

ផ្នែកបង្ហាញ Features ឬ Services ផ្តល់ជូន ។

📬 Contact Section

ផ្នែក Form ទំនាក់ទំនង ស្ថិតនៅ Bottom ។

Single Page Layout ងាយ Maintain ព្រោះ File HTML តែមួយ ។ ក៏ប៉ុន្តែ បើ Content ច្រើន Loading Time ចាប់ផ្តើម (Initial Load) នឹងយឺត ។ ដំណោះស្រាយ គឺ Lazy Load Images, Code Splitting ។ ការប្រើ `<section id="about">` ជា Semantic Tags ជួយ Structure ។

ឧទាហរណ៍ – SINGLE PAGE LAYOUT

ឧទាហរណ៍ Single Page Layout ជាមួយ Semantic

ឧទាហរណ៍ Single Page Layout ដោយប្រើ Semantic Tags ។ Navigation ប្រើ Anchor Link ចូល Section នីមួយៗ ។ `scroll-behavior: smooth` ធ្វើ Smooth Scroll ។

```
<!-- HTML — Single Page Layout with Semantic Tags -->
<body>

  <!-- Navigation ទំព័រ -->
  <header>
    <nav>
      <ul>
        <li><a href="#hero">ទំព័រដើម</a></li>
        <li><a href="#about">អំពីយើង</a></li>
        <li><a href="#services">សេវាកម្ម</a></li>
        <li><a href="#contact">ទំនាក់ទំនង</a></li>
      </ul>
    </nav>
  </header>

  <!-- Sections ចម្បង -->
  <main>
    <section id="hero">
      <h1>ស្វាគមន៍មកកាន់ Website</h1>
      <p>Hero Tagline</p>
    </section>

    <section id="about">
      <h2>អំពីយើង</h2>
      <p>ខ្លឹមសារ Story...</p>
    </section>

    <section id="services">
      <h2>សេវាកម្មរបស់យើង</h2>
      <article>...</article>
    </section>

    <section id="contact">
      <h2>ទំនាក់ទំនង</h2>
      <form>...</form>
    </section>
  </main>

  <footer>
    <p>&copy; 2024</p>
  </footer>

</body>

/* CSS — Single Page */
html {
  scroll-behavior: smooth;
}
section {
  min-height: 100vh;
  padding: 60px 20px;
}
header {
  position: fixed;
  top: 0;
  width: 100%;
  z-index: 1000;
}
```

ចំណាំ: `id` លើ `<section>` ប្រើ ជា Target Anchor Link (`href="#about"`) ។ `scroll-behavior: smooth` ធ្វើ Animation Smooth ពេល User Click Link ។

ឧទាហរណ៍ – SINGLE PAGE LAYOUT

ឧទាហរណ៍ Single Page Layout ជាមួយ Non-Semantic

ឧទាហរណ៍ Single Page Layout ដោយប្រើ **Non-Semantic Div Tags** ។ Logic ដូចគ្នា ប៉ុន្តែ `<section>` ត្រូវ Replace ដោយ `<div>` ។

```
<!-- HTML — Single Page Layout with Non-Semantic Tags -->
<body>

  <div class="header">
    <div class="nav">
      <ul>
        <li><a href="#hero">ទំព័រដើម</a></li>
        <li><a href="#about">អំពីយើង</a></li>
        <li><a href="#services">សេវាកម្ម</a></li>
        <li><a href="#contact">ទំនាក់ទំនង</a></li>
      </ul>
    </div>
  </div>

<div id="hero" class="section">
  <h1>ស្វាគមន៍មកកាន់ Website</h1>
  <p>Hero Tagline</p>
</div>

<div id="about" class="section">
  <h2>អំពីយើង</h2>
  <p>ខ្លឹមសារ Story...</p>
</div>

  <div id="services" class="section">
    <h2>សេវាកម្មរបស់យើង</h2>
  </div>

  <div id="contact" class="section">
    <h2>ទំនាក់ទំនង</h2>
  </div>

  <div class="footer">
    <p>&copy; 2024</p>
  </div>

</body>

/* CSS — Single Page (Non-Semantic) */
 html { scroll-behavior: smooth; }
 .section { min-height: 100vh; padding: 60px 20px; }
 .header {
   position: fixed;
   top: 0; width: 100%;
   z-index: 1000;
 }
```

Semantic (ណែនាំ)

ប្រើ `<section id="about">` – Browser, SEO ដឹងថា Section About

Non-Semantic

ប្រើ `<div id="about">` – Work ដូចគ្នា ប៉ុន្តែ Browser ចាត់ជា Div ធម្មតា

ប្លង់ ៣.៥

អំពី Grid Layout

CSS Grid Layout គឺ System Layout ដ៏មានឥទ្ធិពល ក្នុង CSS ទំនើប ដែលអនុញ្ញាត ឱ្យ Developer បង្កើត Layout ២ D (ទំហំ Row និង Column) ដ៏ស្មុគ្រស្មាញ ដោយ Code ងាយ ។ Grid ផ្តល់ Control ច្បាស់លាស់ លើ Position, Size, Alignment ។

Property ចម្បង `display: grid` ចាប់ Grid Container ។ `grid-template-columns` កំណត់ Columns ។ `grid-template-rows` កំណត់ Rows ។ `gap` ចន្លោះ ។ `grid-column: span 2` ឱ្យ Element ពន្លា ២ Columns ។ ការប្រើ `fr` Unit (Fractional) ធ្វើ Fluid Columns ។

grid-template-columns

កំណត់ Column ចំនួន ហើយទំហំ ។ `repeat(3, 1fr)` = 3 Columns ស្មើ ។

grid-template-areas

ដាក់ ឈ្មោះ Zone ដែល Element ត្រូវ Fill ។ ធ្វើ Visual Layout ច្បាស់ ក្នុង CSS ។

gap

ចន្លោះ Row និង Column ។ ប្រើ `gap: 20px` ឬ `gap: 10px 20px` ។

Grid Layout ខ្លាំងជាង Flexbox ក្នុងការគ្រប់គ្រង 2D Layout ។ Flexbox ល្អ 1D (Row ឬ Column) ។ ការ Combine Grid (Outer Layout) + Flexbox (Inner Components) គឺ Pattern ពេញនិយមបំផុតក្នុង Modern Web Design ។ Grid Support Browser ចាប់ IE 11+ Chrome, Firefox, Safari ទំនើប ។

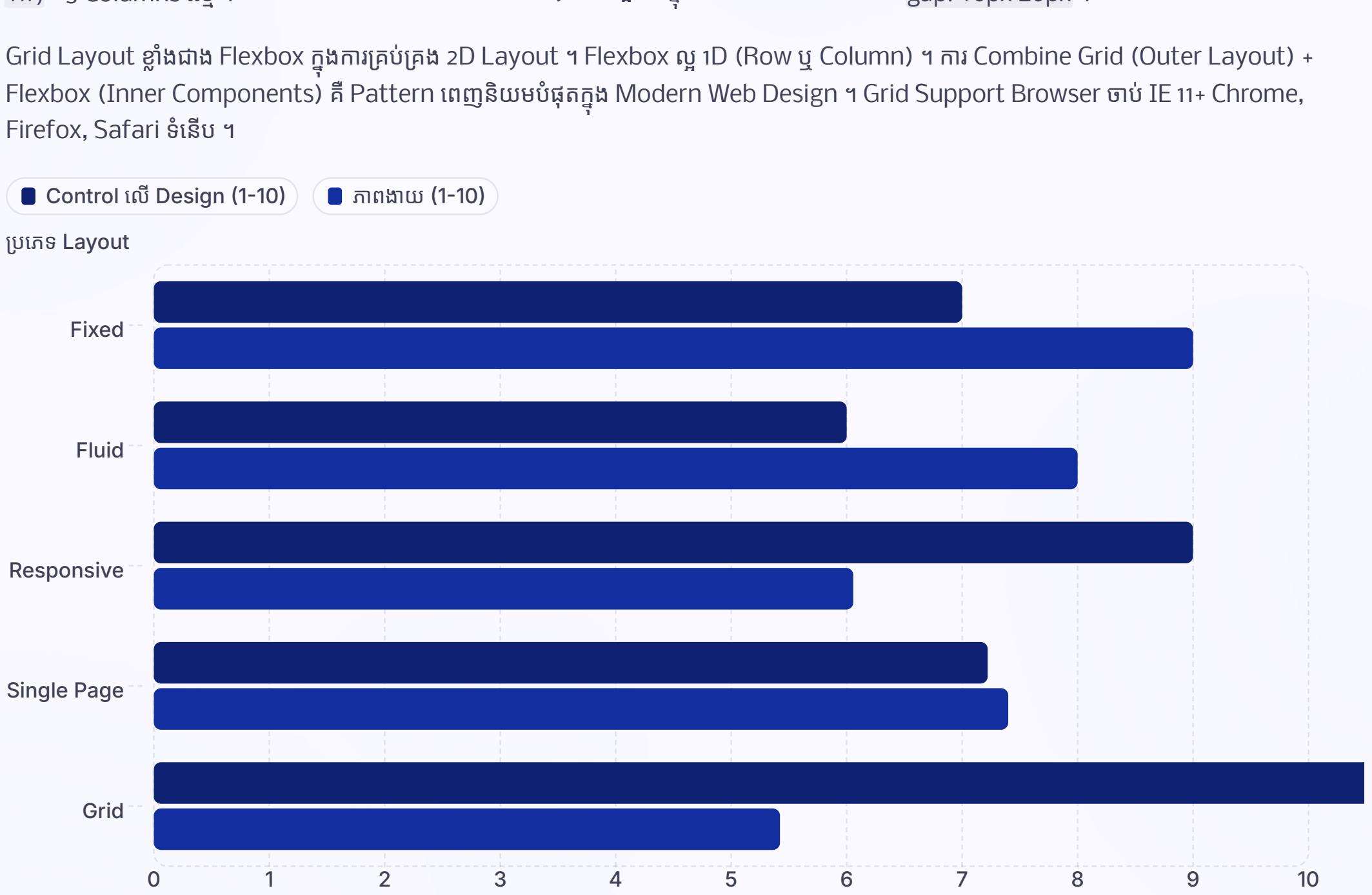


Chart ខាងលើ ប្រៀបធៀប Control (ការគ្រប់គ្រង Design) និង ភាពងាយ (Ease of Use) ក្នុងប្រភេទ Layout ទាំង 5 ។ Grid ផ្តល់ Control ខ្ពស់ (10/10) ប៉ុន្តែ មានភាពស្មុគ្រស្មាញ ។ Fixed ងាយ ប៉ុន្តែ Control ទាប ។

ឧទាហរណ៍ – GRID LAYOUT

ឧទាហរណ៍ Grid Layout ជាមួយ Semantic

ឧទាហរណ៍ **CSS Grid Layout** ដែលប្រើ Semantic Tags ។ `grid-template-areas` ធ្វើ Visual Map ក្នុង CSS ហើយ Property `grid-area` ដាក់ Element ទៅ Zone ត្រូវ ។

```
<!-- HTML — Grid Layout with Semantic Tags -->
<body class="page-grid">

  <header class="area-header">
    <h1>Website Title</h1>
  </header>

  <nav class="area-nav">
    <ul>
      <li><a href="#">ទំព័រដើម</a></li>
      <li><a href="#">Blog</a></li>
    </ul>
  </nav>

  <main class="area-main">
    <article>
      <h2>ចំណងជើង</h2>
      <p>មាតិកា...</p>
    </article>
  </main>

  <aside class="area-sidebar">
    <h3>Sidebar</h3>
    <p>ព័ត៌មានបន្ថែម...</p>
  </aside>

  <footer class="area-footer">
    <p>&copy; 2024</p>
  </footer>

</body>

/* CSS — Grid Layout with Named Areas */
.page-grid {
  display: grid;
  grid-template-columns: 200px 1fr;
  grid-template-rows: auto auto 1fr auto;
  grid-template-areas:
    "header  header"
    "nav     nav"
    "sidebar main"
    "footer  footer";
  min-height: 100vh;
  gap: 10px;
}

.area-header  { grid-area: header;  }
.area-nav     { grid-area: nav;     }
.area-main    { grid-area: main;    }
.area-sidebar { grid-area: sidebar; }
.area-footer  { grid-area: footer;  }
```

គន្លឹះ: `grid-template-areas` ធ្វើ ASCII Map ក្នុង CSS ។ Developer ម្នាក់ ម្នាក់ អានហើយ ភ្លាមៗ យល់ Layout ទំព័រ – ជា Feature ដ៏ Powerful ។

ឧទាហរណ៍ – GRID LAYOUT

ឧទាហរណ៍ Grid Layout ជាមួយ Non-Semantic

ឧទាហរណ៍ CSS Grid Layout ដែលប្រើ **Non-Semantic Div Tags** ។ CSS Grid Logic ដូចគ្នា ប៉ុន្តែ HTML Structure ប្រើ `<div>` ទាំងអស់ ។

```
<!-- HTML — Grid Layout with Non-Semantic Tags -->
<body class="page-grid">

  <div class="area-header">
    <h1>Website Title</h1>
  </div>

  <div class="area-nav">
    <ul>
      <li><a href="#">ទំព័រដើម</a></li>
      <li><a href="#">Blog</a></li>
    </ul>
  </div>

  <div class="area-main">
    <h2>ចំណងជើង</h2>
    <p>មាតិកា...</p>
  </div>

  <div class="area-sidebar">
    <h3>Sidebar</h3>
    <p>ព័ត៌មានបន្ថែម...</p>
  </div>

  <div class="area-footer">
    <p>&copy; 2024</p>
  </div>

</body>

/* CSS — Grid Layout (Non-Semantic) — CSS ដូចគ្នា */
.page-grid {
  display: grid;
  grid-template-columns: 200px 1fr;
  grid-template-rows: auto auto 1fr auto;
  grid-template-areas:
    "header  header"
    "nav     nav"
    "sidebar main"
    "footer  footer";
  min-height: 100vh;
  gap: 10px;
}
.area-header  { grid-area: header;  }
.area-nav     { grid-area: nav;     }
.area-main    { grid-area: main;    }
.area-sidebar { grid-area: sidebar; }
.area-footer  { grid-area: footer;  }
```

🏆 សង្ខេប

CSS Grid ដូចគ្នា Semantic ឬ Non-Semantic ។ ការជ្រើស Tag ប៉ះពាល់ SEO, Accessibility មិនប៉ះ Visual ។

✅ ការណែនាំ

ប្រើ Semantic Tags ជានិច្ច (`<header>`, `<main>`, `<footer>`) ហើយ ប្រើ `<div>` ក្នុងករណីពិសេស ។

🚀 ជំហានបន្ទាប់

ហ្វឹកហ្វឺន Code Grid Layout ដោយ Practice Project ជាក់ស្តែង ។ ប្រើ Browser DevTools Grid Inspector ។

សេចក្តីសន្និដ្ឋាន: ការយល់ដឹងអំពី Semantic vs Non-Semantic Elements និងប្រភេទ Layout (Fixed, Fluid, Responsive, Single Page, Grid) ជា Foundation ដ៏ចាំបាច់ ក្នុងការក្លាយជា Web Developer ដ៏ជំនាញ ។ ចូរ Practice Code និងបង្កើត Project ជាក់ស្តែង ដើម្បីបង្រួម (Solidify) ចំណេះដឹងទាំងនេះ ។